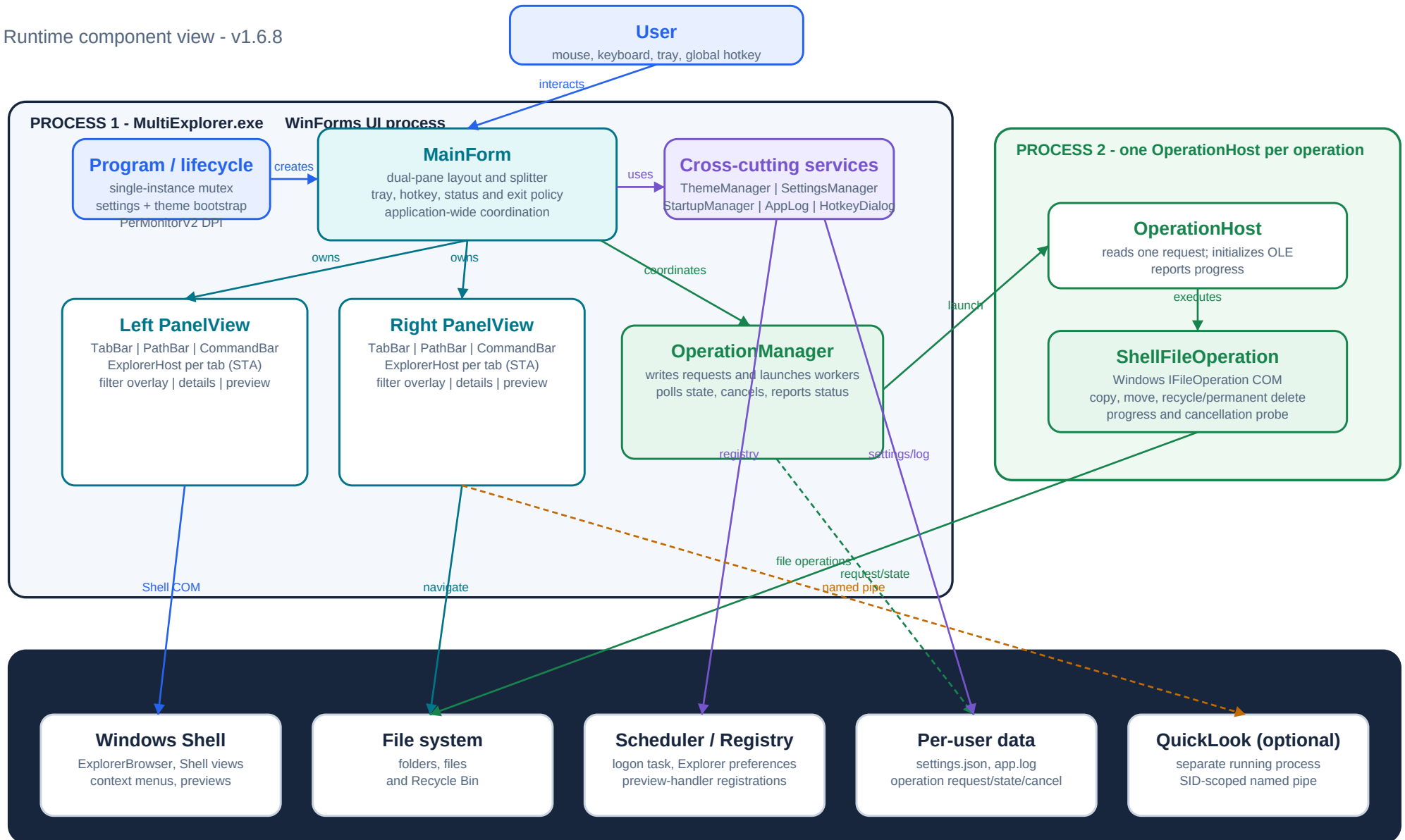


MultiExplorer Application Architecture

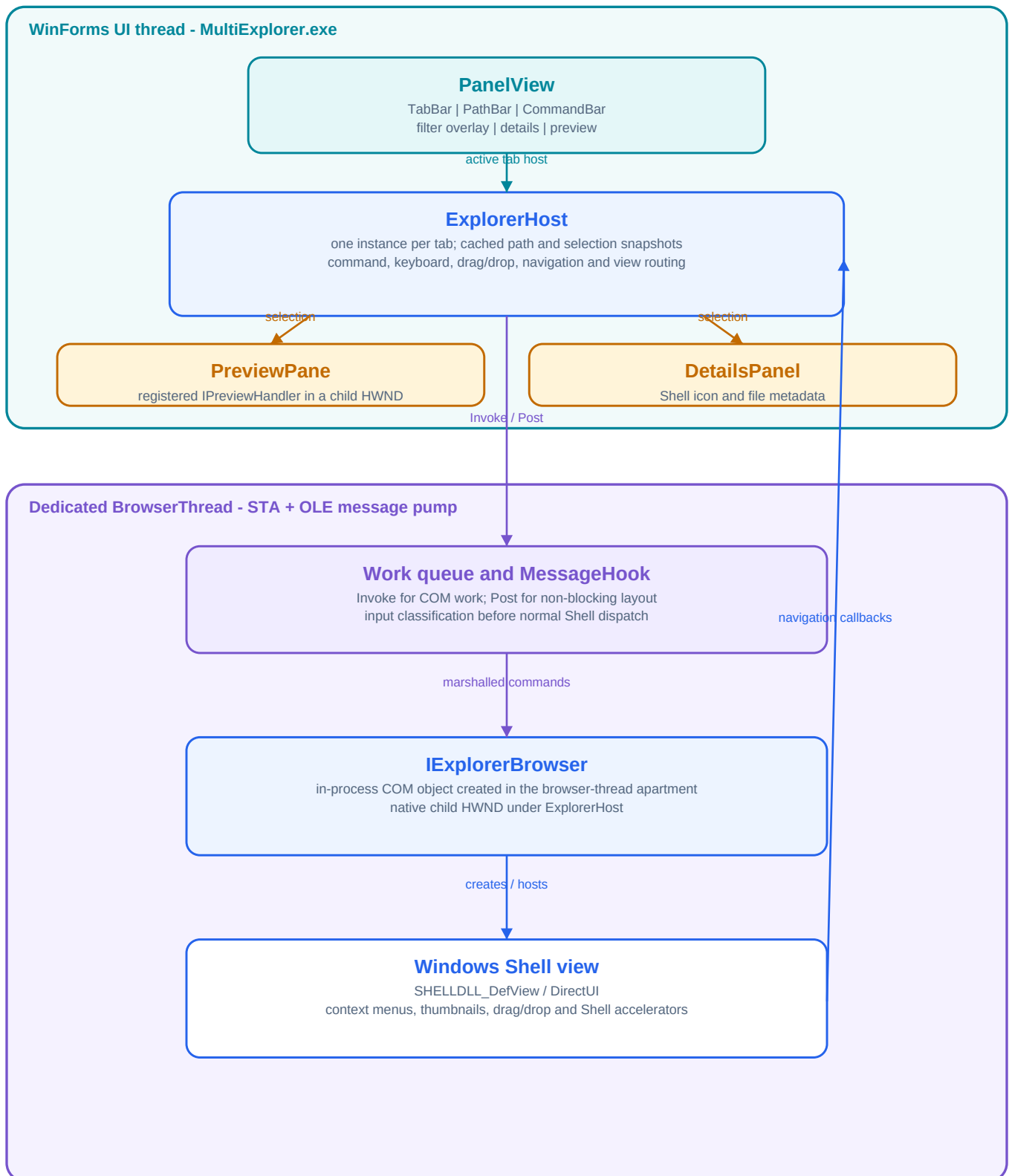
Runtime component view - v1.6.8



Solid arrows: direct calls / ownership. Dashed arrows: asynchronous file or pipe exchange.

Explorer hosting and thread model

One dedicated OLE STA and native Shell view per ExplorerHost



Runtime composition and Explorer hosting

Process boundaries, UI ownership, and Shell thread affinity

1. Application bootstrap and lifetime

Program acquires a named single-instance mutex before any UI exists. A later interactive launch broadcasts a registered Windows message so the existing application restores and activates itself; duplicate Windows-startup launches remain silent. StartupManager registers a zero-delay, current-user Task Scheduler logon trigger and removes the obsolete Run entry. A logon launch exposes the tray icon first and defers native Explorer panes until the user opens the window. Settings and theme load before WinForms creates **MainForm** on the UI STA.

2. Dual-pane ownership

MainForm owns the two independent **PanelView** controls, custom splitter, tray icon, status bar, global hotkey, persistence, and file-operation exit policy. A panel owns its tabs, breadcrumb, command bar, filter overlay, details pane, preview pane, and one **ExplorerHost** per tab. Only the active host is visible. The Open in other pane action passes the current folder to the opposite panel; it does not copy anything.

3. Dedicated Shell STA

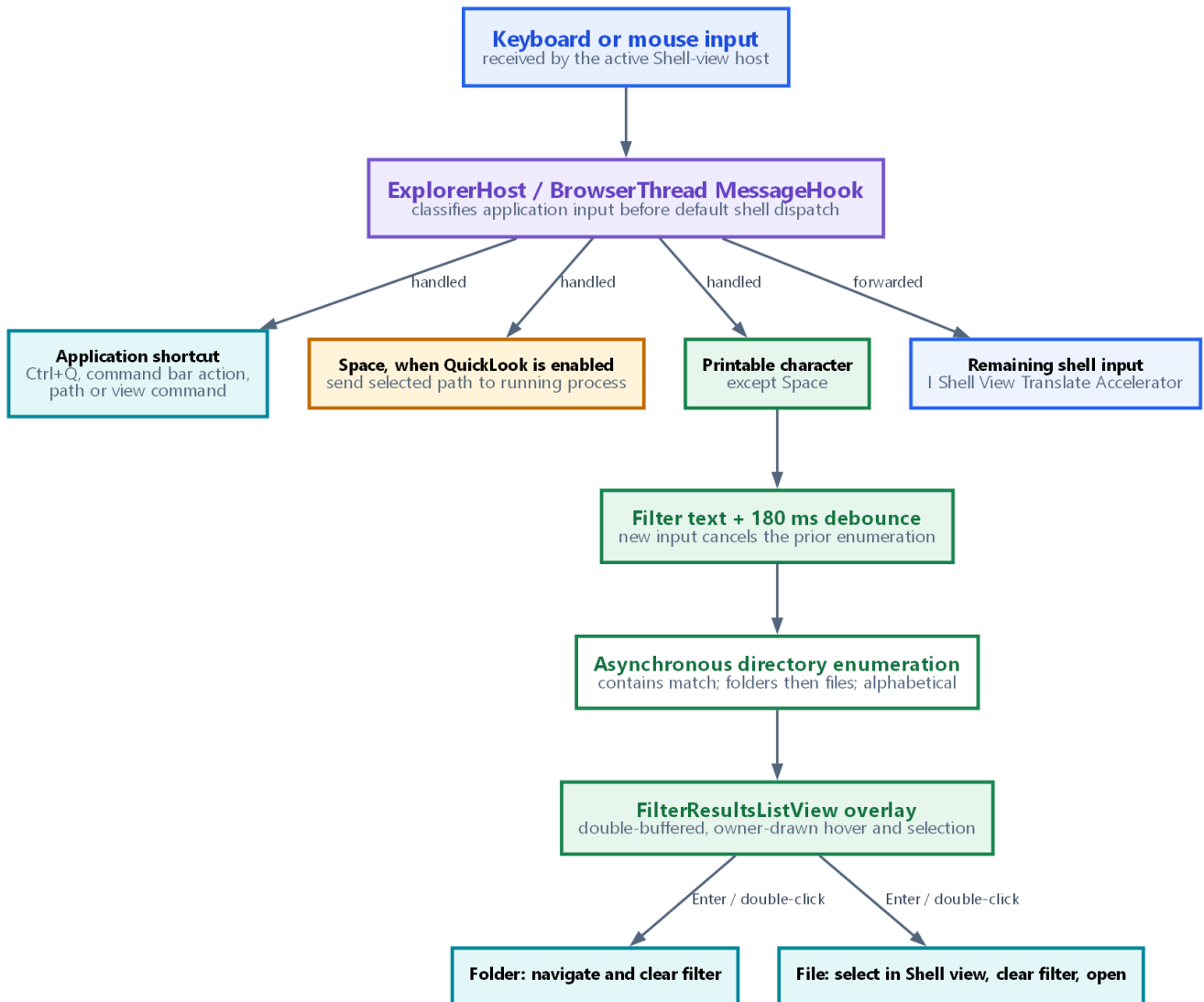
Each ExplorerHost owns a BrowserThread with an OLE-initialized STA and message pump. IExplorerBrowser and its native Shell child window live there, not on the WinForms UI thread. Invoke marshals COM work when a result is needed; Post performs non-blocking work such as bounds changes. This isolates Shell modal loops during drag/drop or a Shell operation and preserves responsiveness of the managed UI.

4. Native functionality

ExplorerHost routes Shell accelerators, context menus, drag/drop, view options, navigation-pane synchronization, selection changes, and QuickLook requests. Navigation callbacks refresh cached paths so the UI does not need repeated synchronous cross-thread COM reads. DetailsPanel uses Shell icons and metadata; PreviewPane discovers a registered IPreviewHandler and hosts it in a child window.

Input and type-to-filter flow

Managed filtering and application shortcuts layered over the active Shell view



Input, navigation, filtering, and selection

Managed behavior layered over the active native Shell view

1. Input ownership

BrowserThread's MessageHook classifies input before normal message dispatch. Application commands are raised to the managed command layer. Space can target a running QuickLook instance only when that option is enabled. Other printable characters begin the contains filter, while standard Explorer keys are forwarded through `IShellView::TranslateAccelerator`.

2. Type-to-filter

PanelView waits 180 ms after the latest character, cancels earlier work, and asynchronously enumerates the current directory. Folder matches precede file matches and both are alphabetical. The `FilterResultsListView` overlay is double-buffered and owner-drawn, so hover feedback redraws only the affected rows instead of repainting the full list.

3. Preserving selection

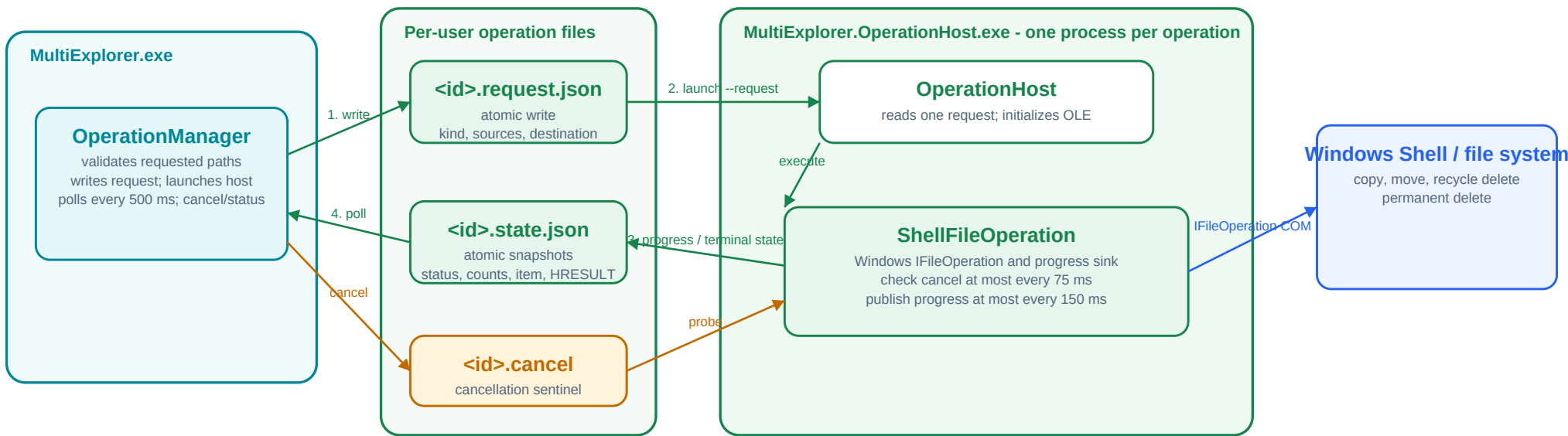
Opening a matched folder navigates and clears the overlay. Opening a matched file first selects that path in the underlying `ExplorerHost`, then clears the overlay and invokes the default file handler. Navigation callbacks update cached path information, and `MainForm` polls paths every 300 ms so tab labels, path bars, persistence, and optional selection panes remain current.

4. Stable presentation

The result list overlays the native Shell content but the yellow filter bar is kept outside the host container. Native child windows therefore cannot paint over the text entry controls. Hover handling invalidates only the old and new rows, preserving the expected Explorer-style rollover without flickering the entire filtered list.

File-operation IPC

Durable file handoff between the UI and one isolated worker process



Atomic replacement ensures readers see complete JSON documents. The worker writes the authoritative terminal state.

Isolated file operations and IPC

One worker process per copy, move, recycle-delete, or permanent-delete request

1. Durable process handoff

OperationManager validates the requested paths, allocates a GUID, atomically writes a request JSON file, starts MultiExplorer.OperationHost.exe with that request path, and records a queued state. The worker initializes OLE, creates Windows IFileOperation, and reports its own process ID and progress through atomic state snapshots.

2. Progress, cancellation, and failure detection

The worker's progress sink publishes state no more often than every 150 ms except for terminal or failure updates. OperationManager polls every 500 ms, updates the status and tray UI, and marks an operation failed if a worker disappears without the required terminal state. Cancellation creates an id-specific sentinel file; a worker checks it at most every 75 ms and asks IFileOperation to abort safely.

3. Shell fidelity and containment

The worker delegates copy, move, recycle delete, and permanent delete to native IFileOperation. This retains Windows conflict handling and file-operation semantics while preventing worker lifetime or a native modal loop from taking down the interactive explorer window.

4. Terminal states

The shared contract distinguishes Queued, Running, Cancelling, Completed, Failed, and Cancelled states. The terminal state is the authoritative worker outcome and includes any Shell HRESULT and message needed for diagnostics. The file-based exchange permits the UI and worker processes to be restarted or observed independently.

Persistence, integrations, and design decisions

Local per-user state and feature-detected Windows integrations

Integration	Purpose
%APPDATA%\MultiExplorer\settings.json	Persisted window, panel, tab, path history, tray, theme, hotkey, startup, and QuickLook state.
%LOCALAPPDATA%\MultiExplorer\app.log	Rolling diagnostic log. Warnings and errors also reach the status bar.
%LOCALAPPDATA%\MultiExplorer\Operations	Atomic request, state, and cancellation files shared with operation-host processes.
Windows Task Scheduler	Current-user, zero-delay logon task used for prompt sign-in launch.
Current-user registry	Explorer display preferences plus deletion of the obsolete Run entry during startup migration.
Classes-root registry	Preview-handler discovery by file extension or ProgID.
Windows Shell COM	Explorer views, thumbnails, context menus, drag/drop, preview handlers, and file operations.
QuickLook named pipe	Optional outbound message carrying the selected path to a running QuickLook process.

Architecture considerations

Concern	Implementation consequence
Responsiveness	The UI STA coordinates managed controls; each native Explorer view has a dedicated STA. Filtering is asynchronous, debounced, and cancellable.
Fault containment	Shell file operations run in a separate executable. Stale state and an unexpectedly exited worker become an explicit failure.
Windows fidelity	Navigation, icons, menus, drag/drop, previews, and file operations intentionally delegate to Windows Shell interfaces and installed extensions.
Theme and DPI	ThemeManager combines a managed palette with native theme treatment. Shell views are recreated after live theme changes; PerMonitorV2 keeps host and child rendering aligned.
Optional dependency	QuickLook is feature-detected. A missing process or pipe produces a targeted message without affecting the core explorer.

Component reference and maintenance

Source-level responsibility map

Component	Source	Runtime responsibility
Program	Program.cs	Bootstrap, single instance, settings/theme loading, DPI mode, and message loop.
MainForm	MainForm.cs	Top-level window, dual panes, splitter, tray, hotkey, status, exit policy, and operation coordination.
PanelView	PanelView.cs	Per-pane composition, tab ownership, filtering, optional panes, and command dispatch.
ExplorerHost / BrowserThread	ExplorerHost.cs / BrowserThread.cs	Per-tab native Shell hosting, input routing, navigation, selection, and dedicated STA/OLE message pump.
PathBar / TabBar / CommandBar	PathBar.cs / TabBar.cs / CommandBar.cs	Managed navigation, tab UI, Shell/application commands, and custom layered popups.
DetailsPanel / PreviewPane	DetailsPanel.cs / PreviewPane.cs	Metadata and preview-handler hosting for the selected item.
OperationManager / contracts	OperationManager.cs / OperationContracts.cs	Worker launch, state polling, cancellation, JSON models, and atomic store.
OperationHost / ShellFileOperation	MultiExplorer.OperationHost/	Per-operation OLE worker and IFileOperation execution.
ThemeManager	ThemeManager.cs	Palette, managed menu renderer, and native dark-mode application.
SettingsManager / StartupManager / AppLog	SettingsManager.cs / StartupManager.cs / AppLog.cs	Per-user persistence, logon-task registration, and rolling diagnostic logging.
NativeMethods	NativeMethods.cs	P/Invoke and COM interop declarations.

Maintenance rule

Update this document, the Markdown reference, and the four Graphviz DOT diagrams whenever a process boundary, IPC contract, storage location, Windows Shell integration point, thread ownership rule, or major UI responsibility changes. Routine visual changes normally require only a note update unless they alter one of these boundaries.